

Poné el proyecto HIS a andar en tu máquina

Esta guía es para el resto del equipo: qué instalar y qué configurar en Windows para clonar el proyecto, levantar el backend y el frontend, y tener la base de datos corriendo en local. Usamos **PostgreSQL instalado nativo** (no Docker) — así evitamos depender de virtualización en máquinas que capaz no la soportan bien.

Repo: **Tp-final-metologia-y-programacion---grupo-3**

Backend: **NestJS + TypeORM**

Frontend: **Vue 3 + Vite**

DB: **PostgreSQL 17 (nativo)**

Qué necesitás instalar

Cuatro programas. Los dos primeros seguro los tenés si ya laburás con git/node; los otros dos son específicos de este proyecto.

Git **obligatorio**

Para clonar el repositorio y subir tus cambios.

git-scm.com/downloads

Node.js **obligatorio**

Versión LTS, 20 o superior (usamos 22 sin problema).
Trae npm incluido.

nodejs.org

PostgreSQL **obligatorio** 17

El motor de base de datos.
Instalación nativa para Windows, no el contenedor Docker.

postgresql.org/download/window

DataGrip **recomendado**

Cliente visual para ver tablas y correr SQL. Gratis con uso no comercial (licencia de JetBrains).

jetbrains.com/datagrip

NOTA

DataGrip es opcional — es solo un visor. Si preferís, podés usar **pgAdmin** (que se instala junto con PostgreSQL, gratis) o la consola `psql` por terminal. Los pasos de este documento asumen DataGrip porque es lo que usamos nosotros.

Instalación paso a paso

En este orden. Cada paso depende del anterior.

1

Git

Instalador estándar, dejá todas las opciones por defecto ("Next" hasta el final). Al terminar, verificá en una terminal (PowerShell o cmd):

```
git --version
```

2

Node.js

Bajá el instalador **LTS** del sitio oficial. Durante la instalación no hace falta tildar "Tools for Native Modules". Verificá:

```
node --version  
npm --version
```

3

PostgreSQL (instalación nativa)

Corré el instalador de postgresql.org. Cuando te pida:

- **Password del superusuario "postgres":** elegí una contraseña y anotala — la vas a necesitar en el .env del backend en el próximo paso. No hace falta que sea igual a la de tus compañeros, cada uno usa la suya en su propia máquina.
- **Puerto:** dejalo en el default, 5432.
- **Locale:** dejalo en el default.
- **Stack Builder** (aparece al final): se puede cerrar/saltar, no lo necesitamos.

A DIFERENCIA DE DOCKER

La instalación nativa **no crea la base del proyecto sola**. Eso lo hacemos a mano en el paso 5.

4

DataGrip — conectar al servidor

Abrí DataGrip → **New** → **Data Source** → **PostgreSQL**. Completá:

CAMPO	VALOR
Host	localhost
Port	5432
User	postgres
Password	la que pusiste en el paso 3

Dale a **Test Connection** — tiene que tildar en verde — y despues **OK**. La primera vez te va a ofrecer descargar el driver de PostgreSQL: aceptá.

5

Crear la base de datos HospitalHIS

Clic derecho sobre postgres@localhost en el árbol de la izquierda → **New** → **Database**.

EL ERROR MÁS COMÚN

Postgres distingue mayúsculas de minúsculas en el nombre de la conexión (no en las consultas SQL comunes, pero sí acá). Si escribís HospitalHIS sin comillas, Postgres puede guardarla como hospitalhis y el backend no la va a encontrar, tirando el error no existe la base de datos «HospitalHIS».

La forma segura: abrí una consola SQL sobre postgres@localhost (ícono de rayo o Ctrl+Shift+F10) y corré:

```
CREATE DATABASE "HospitalHIS";
```

Las comillas dobles fuerzan que quede exactamente así, con esas mayúsculas.

Refrescá el árbol (ícono circular arriba, o clic derecho → Refresh) y confirmá que HospitalHIS aparece como hermana de postgres.

Clonar y configurar el proyecto

6

Cloná el repositorio

En la carpeta donde quieras tener el proyecto:

```
git clone https://github.com/Fabian-Craver0/Tp-final-metologia-y-programacion---  
cd Tp-final-metologia-y-programacion---grupo-3
```

7

Instalá las dependencias

Son dos proyectos separados (backend/ y frontend/), cada uno con su propio `package.json`:

```
cd backend  
npm install  
  
cd ../frontend  
npm install
```

Creá los archivos `.env`

Ninguno se sube al repo (están en `.gitignore`) — cada uno crea los suyos a partir del `.env.example` correspondiente. Con la instalación nativa necesitás **dos** archivos:

backend/.env — copió backend/.env.example como backend/.env y completá:

VARIABLE	VALOR
PORT	3000
DB_HOST	localhost
DB_PORT	5432
DB_USERNAME	postgres
DB_PASSWORD	la contraseña que pusiste al instalar Postgres (paso 3)
DB_NAME	HospitalHIS
JWT_SECRET	cualquier texto que quieras — no hace falta que coincida entre compañeros
JWT_EXPIRES_IN	8h

frontend/.env — copió frontend/.env.example como frontend/.env:

```
VITE_API_URL=http://localhost:3000
```

SOBRE EL `.ENV` DE LA RAÍZ

Existe un tercer `.env.example` en la raíz del proyecto, pero es solo para quien prefiera levantar la base con docker compose en vez de instalación nativa. Si seguiste esta guía, no lo necesitás.

Levantar el proyecto

Con Postgres corriendo (arranca solo como servicio de Windows al prender la PC) y los `.env` creados, ya podés levantar backend y frontend.

9

Backend

En una terminal, parado en backend/:

```
npm run start:dev
```

Si todo está bien vas a ver `Nest application successfully started` al final del log. Quedan disponibles:

- API: **`http://localhost:3000`**
- Swagger (documentación interactiva de la API): **`http://localhost:3000/api`**

La primera vez que conecta, TypeORM crea solo las tablas que hagan falta a partir del código (por ahora, `synchronize: true` — es algo temporal mientras recién arrancamos, ver nota más abajo).

10

Frontend

En **otra** terminal (dejá la del backend corriendo), parado en frontend/:

```
npm run dev
```

Vite te va a imprimir la URL local, normalmente **`http://localhost:5173`**.

SOBRE SYNCHRONIZE: TRUE

Ahora mismo el backend está configurado para crear/ajustar las tablas solo, comparando contra las entities del código — cómodo mientras estamos armando la base, pero riesgoso en equipo: si dos personas cambian entities distintas y ambas corren el backend, se pueden pisar cambios de tabla sin darse cuenta. Antes de laburar todos en paralelo con datos reales vamos a pasar a migraciones (queda documentado en la sección 12 del informe de arquitectura). Si en tu `npm run start:dev` ves que se borró o cambió una tabla que no tocaste vos, avisá en el grupo antes de seguir.

Problemas comunes

Los tres que más aparecieron armando esto.

`error: no existe la base de datos «HospitalHIS»`

El nombre de la base que ves en DataGrip no coincide, letra por letra, con `DB_NAME` del `.env`. Revisá el paso 5: la base tiene que llamarse exactamente `HospitalHIS` (creada con comillas dobles para respetar las mayúsculas).

`password authentication failed for user "postgres"`

`DB_PASSWORD` en `backend/.env` no coincide con la contraseña que le pusiste al usuario `postgres` al instalar PostgreSQL en **tu** máquina. No tiene que ser la misma que la de tus compañeros — cada uno tiene su propia instalación local.

`ECONNREFUSED` o no conecta a nada en el puerto 5432

Puede haber más de un Postgres escuchando ese puerto (por ejemplo, un contenedor Docker viejo además de la instalación nativa). Revisá con:

```
netstat -ano | findstr 5432
```

Si aparece más de un proceso, o Docker Desktop está corriendo con un contenedor de Postgres, pausalo o bajalo y probá de nuevo.

Para seguir

- **Arquitectura del proyecto** (capas, patrones, por qué está organizado así): informe de arquitectura del equipo.
- **Dudas de setup que no están acá:** preguntá en el grupo antes de tocar la config de otro — casi todo lo de este documento salió de problemas reales que ya pisamos.

